

Student ID:

IT24103352



BSc (Hons) in Information Technology

IT3091 - Machine Learning

Y3.S1 – 2026

Faculty of Computing

LAB SHEET 06

Regression , SVM & SVR with Scikit-Learn

Learning Outcomes

By the end of this lab session, students will be able to:

- Implement Simple Linear Regression from scratch using NumPy.
- Calculate predictions, residuals, Mean Squared Error (MSE), and regression evaluation metrics.
- Train a Linear Regression model from scratch using Gradient Descent.
- Fit and tune a Support Vector Machine (SVM) classifier using Scikit-Learn.
- Explain the decision boundary, margin, support vectors, and Hinge Loss used in SVM, and relate them to a fitted model's own attributes.
- Fit and tune a Support Vector Regression (SVR) model using Scikit-Learn.
- Explain the ϵ -tube and ϵ -insensitive loss used in SVR, and how ϵ and C shape the fitted model.
- Compare linear and nonlinear (kernel-based) SVM/SVR behaviour, and relate the library-based models to the from-scratch regression model.

Software / Resources Required

- Python 3.x

- Jupyter Notebook / Google Colab
- NumPy, Pandas, Matplotlib and Scikit-learn

Important: Part A (Linear Regression) is still implemented from scratch using basic Python/NumPy operations, exactly as before. Parts B and C (SVM and SVR) now use Scikit-Learn directly: you will fit real models, inspect their attributes, and explore how their hyperparameters change behaviour, rather than deriving the training loop by hand.

Part A

Linear Regression from Scratch

A.1 – Dataset & Problem Setup

A university wants to predict a student's **Exam Score** based on the number of **Hours Studied**.

Student	Hours Studied (X)	Exam Score (Y)
A	1	48
B	2	52
C	3	58
D	4	64
E	5	70
F	6	75

```
import numpy as np
```

```
X = np.array([1, 2, 3, 4, 5, 6])
```

```
y = np.array([48, 52, 58, 64, 70, 75])
```

Task 1 – Understand the Problem

1. Identify the input feature, target variable, and type of supervised learning problem.
2. Create a scatter plot of Hours Studied vs. Exam Score.
3. Describe the relationship visible in the scatter plot.

4. Why is regression more appropriate than classification for this problem?

1.

Input feature: Hours Studied (X).

Target variable: Exam Score (Y).

Type of supervised learning: Regression.

2.

Scatter plot should show points roughly rising from left to right.

3.

The relationship is positive and approximately linear: as hours studied increase, exam score tends to increase.

4.

Regression is more appropriate because the target is a continuous numerical value, not a discrete class label.

A.2 – Build the Prediction Function

Simple Linear Regression

$$\hat{y} = \beta_0 + \beta_1 x$$

β_0 = intercept β_1 = regression coefficient $\hat{y}$ = predicted value

Task 2 – Initial Predictions

1. Initialize $\beta_0 = 0$ and $\beta_1 = 0$.
2. Complete the prediction function without using Scikit-learn.
3. Generate the initial predictions for all observations.
4. Explain the roles of β_0 and β_1 .

```
def predict(X, beta0, beta1):  
    # TODO: calculate y_hat  
    pass
```

```
beta0 = 0
```

```
beta1 = 0
```

```
y_hat = predict(X, beta0, beta1)
```

4.

β_0 (intercept): predicted value when $X = 0$.

β_1 (slope/coefficient): change in predicted Y for a one-unit increase in X .

A.3 – Residuals & Mean Squared Error

Residual: $e_i = y_i - \hat{y}_i$

Mean Squared Error: $MSE = (1/n) \sum (y_i - \hat{y}_i)^2$

Task 3 – Calculate Error from Scratch

1. Calculate the residual for every observation.
2. Implement MSE from scratch.
3. Display the actual value, predicted value and residual for each observation.
4. What do positive, negative and near-zero residuals indicate?
5. Why are errors squared when calculating MSE?

```
def mse(y, y_hat):  
    # TODO: implement Mean Squared Error  
    pass
```

4.

Positive residual: actual > predicted.

Negative residual: actual < predicted.

Near-zero residual: prediction is close to actual.

5.

Errors are squared in MSE to remove sign and penalise larger errors more strongly.

A.4 – Train Linear Regression using Gradient Descent

$$\partial J / \partial \beta_0 = (2/n) \sum (\hat{y}_i - y_i)$$

$$\partial J / \partial \beta_1 = (2/n) \sum (\hat{y}_i - y_i) x_i$$

Update: $\beta_j := \beta_j - \alpha(\partial J / \partial \beta_j)$

Task 4 – Implement Gradient Descent

1. Set a suitable learning rate and number of iterations.
2. Calculate predictions and gradients at each iteration.
3. Update β_0 and β_1 .
4. Store the MSE at each iteration.
5. Display the final β_0 , β_1 and MSE.
6. Plot Iteration vs. MSE and explain the convergence behaviour.
7. Plot the final regression line together with the observations.

```
learning_rate = 0.01
iterations = 1000
beta0 = 0
beta1 = 0
loss_history = []

for i in range(iterations):
    y_hat = predict(X, beta0, beta1)

    # TODO: calculate dbeta0
    # TODO: calculate dbeta1

    # TODO: update beta0 and beta1

    # TODO: calculate and store MSE
```

Final values are approximately:

$$\beta_0 \approx 41.67$$

$$\beta_1 \approx 5.57$$

$$\text{MSE} \approx 0.27$$

Iteration vs MSE should show a steep drop initially, then flattening as it converges.

A.5 – Evaluate the Regression Model

$$\text{MAE} = (1/n) \sum |y_i - \hat{y}_i|$$

$$\text{RMSE} = \sqrt{\text{MSE}}$$

$$R^2 = 1 - [\sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2]$$

Task 5 – Metrics from Scratch

1. Implement MAE, MSE, RMSE and R^2 without using Scikit-learn metrics.
2. Interpret each value.
3. Which metrics are expressed in the same units as the target?
4. Which metric penalizes larger errors more strongly?
5. Predict the exam score for a student who studies **4.5 hours**.

2.

$$\text{MAE} \approx 0.43$$

$$\text{MSE} \approx 0.27$$

$$\text{RMSE} \approx 0.52$$

$$R^2 \approx 0.997$$

3.

MAE: average absolute error in exam-score points.

RMSE: typical error magnitude, same units as target.

R^2 : about 99.7% of variance explained.

4.

Metrics in same units as target: MAE and RMSE.

MSE is in squared units.

MSE/RMSE penalise larger errors more strongly.

5.

Predicted exam score ≈ 66.74 .

A.6 – Verify using Scikit-learn

1. Train `LinearRegression` using the same dataset.
2. Display the intercept and coefficient.
3. Predict the score for 4.5 hours.
4. Compare the results with your from-scratch model.

Using `LinearRegression`:

Intercept ≈ 41.67

Coefficient ≈ 5.57

Prediction for 4.5 hours ≈ 66.74

This matches the from-scratch gradient descent result closely.

Part B

Support Vector Machine (SVM) with Scikit-Learn

B.1 – Binary Classification Dataset

Use the following two-dimensional dataset to investigate how a linear SVM separates two classes.

Observation	X_1	X_2	Class
A	1	2	-1
B	2	2	-1
C	2	3	-1
D	4	4	+1
E	5	4	+1
F	5	5	+1

```
import numpy as np
```

```
X = np.array([[1, 2], [2, 2], [2, 3], [4, 4], [5, 4], [5, 5]])
y = np.array([-1, -1, -1, 1, 1, 1])
```

Task 6 – Visualize the Classification Problem

1. Plot the two classes using a scatter plot.
2. Can the classes be separated by a straight line?
3. Sketch or plot more than one possible separating line.

2.

Yes, the classes can be separated by a straight line.

3.

Many straight lines can separate the classes. The SVM chooses the one with the maximum margin.

B.2 – SVM Decision Function

$$f(x) = w^T x + b$$

For two features: $f(x) = w_1 x_1 + w_2 x_2 + b$

Predict +1 when $f(x) \geq 0$; otherwise predict -1.

Task 7 – Fit an SVC and Inspect Its Decision Function

1. Fit `SVC(kernel='linear', C=1.0)` on the dataset.
2. Retrieve `svc.coef_` and `svc.intercept_` and use them to compute $f(x) = w^T x + b$ by hand for every observation.
3. Compare your hand-computed scores with `svc.decision_function(X)`, they should match.
4. Compare `svc.predict(X)` with the actual classes.

```
from sklearn.svm import SVC
```

```
svc = SVC(kernel='linear', C=1.0)
svc.fit(X, y)
```

```
w, b = svc.coef_[0], svc.intercept_[0]
scores = X @ w + b          # should match svc.decision_function(X)
preds = svc.predict(X)
```

For SVC(kernel='linear', C=1.0):

$w = [0.8, 0.4]$

$b = -3.8$

B.3 – Margin & Support Vectors

Decision boundary: $w^T x + b = 0$

Margin boundaries: $w^T x + b = +1$ and $w^T x + b = -1$

Margin condition: $y_i(w^T x_i + b) \geq 1$

Task 8 – Investigate the Margin

1. Using the w and b from Task 7, plot the decision boundary ($w^T x + b = 0$) and both margin boundaries ($w^T x + b = \pm 1$).
2. Print `svc.support_vectors_` and mark these points on your plot.
3. Explain why exactly these observations are called **support vectors**.
4. Why does SVM prefer a large margin instead of simply finding any separating line?

They are called support vectors because they lie on the margin boundary and directly determine the position and orientation of the decision boundary.

SVM prefers a large margin because it usually gives better generalisation and robustness to new data.

B.4 – Hinge Loss

$$L_i = \max(0, 1 - y_i f(x_i))$$

Task 9 – Hinge Loss from the Fitted Model

1. Use `svc.decision_function(X)` as $f(x)$ for every observation.
2. Calculate the Hinge Loss $L_i = \max(0, 1 - y_i f(x_i))$ for each observation, one line of NumPy, no loop needed.
3. Compare points outside the margin, inside the margin and incorrectly classified points.
4. Which observations produce zero Hinge Loss? How does that compare with `svc.support_vectors_` from Task 8?

```
scores = svc.decision_function(X)
hinge = np.maximum(0, 1 - y * scores)
```

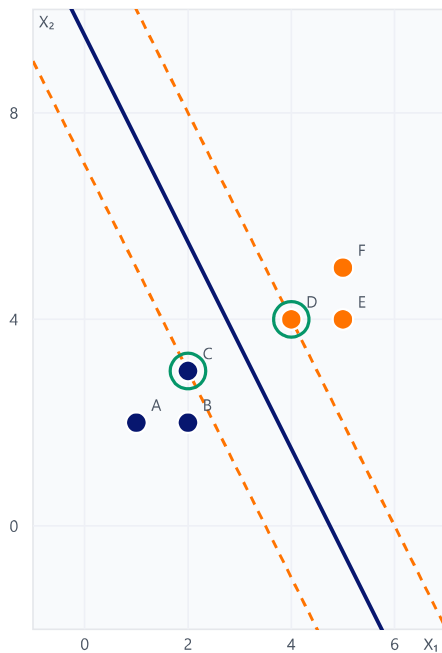
Support vectors are C and D. They produce zero hinge loss because they lie exactly on the margin. Points producing zero hinge loss are not necessarily support vectors; support vectors are the points that constrain the margin.

B.5 – Interactive Sandbox: The Effect of C on the Margin

What C Controls

C is the regularization parameter of `SVC`. A **small C** lets the model accept a wider margin even if that means a few points fall inside it; a **large C** forces the model to fit the training points as tightly as possible, shrinking the margin. Drag the slider below to watch the decision boundary, both margin boundaries and the set of support vectors change, computed from a real `SVC(kernel='linear', C=...)` fit on the exact dataset from B.1.

Task 10 – The Effect of C on the Margin



C	1
Margin width ($2/\ w\ $)	2.24
Support vectors	C, D

Class -1 Class +1 (ringed points are support vectors)

Shown at $C = 1$ (the converged hard-margin fit). Use the slider on screen to see other values of C .

1. Slide C from its smallest to its largest value. What happens to the margin width, and why?
2. How does the number of support vectors change as C increases? Connect this to what a support vector is.
3. Between $C = 0.3$ and $C = 1$ the margin barely changes even though C increases. Why does that happen for this dataset?
4. If you were worried this dataset had a mislabelled point, would you choose a smaller or a larger C ? Explain.

As C increases, the margin width generally decreases. Larger C penalises margin violations more strongly, so the model tries to fit the training data more tightly.

As C increases, the number of support vectors generally decreases. Support vectors are points on or inside the margin. Smaller C allows more points inside the margin.

Once C is large enough, the data is linearly separable, so the model approaches the hard-margin solution. Further increasing C may change the solution only slightly or not at all.

If you suspect a mislabelled point, choose a smaller C , because it allows some margin violations and avoids overfitting to the mislabelled point.

Part C

Support Vector Regression (SVR) with Scikit-Learn

C.1 – Reuse the Regression Dataset

Use the **Hours Studied** → **Exam Score** dataset from Part A. Unlike SVM classification, SVR predicts a continuous value.

Linear SVR Function

$$f(x) = wx + b$$

Instead of trying to minimize every small error, SVR defines an acceptable error region around the regression function.

Task 11 – Fit an SVR and Inspect Its Predictions

1. Fit `SVR(kernel='linear', C=1.0, epsilon=1.0)` on the dataset.
2. Retrieve `svr.coef_` and `svr.intercept_` and use them to compute $f(x) = wx + b$ by hand for every observation.
3. Compare your hand-computed values with `svr.predict(X)`, they should match.
4. Plot the observations together with the fitted line.

```
from sklearn.svm import SVR

svr = SVR(kernel='linear', C=1.0, epsilon=1.0)
svr.fit(X, y)

w, b = svr.coef_[0][0], svr.intercept_[0]
preds = w * X.ravel() + b          # should match svr.predict(X)
```

For `SVR(kernel='linear', C=1.0, epsilon=1.0)`:
 $w = 5.0$

b = 44.0

C.2 – Construct the ϵ -Tube

Upper boundary: $f(x) + \epsilon$ **Lower boundary:** $f(x) - \epsilon$

An observation whose residual falls inside the tube ($|y - f(x)| < \epsilon$) contributes zero loss. scikit-learn calls the observations that end up *on* or *outside* the tube boundary the **support vectors**.

Task 12 – Visualize the ϵ -Tube

1. Using the w and b from Task 11, and $\epsilon = 1.0$, compute the upper boundary $f(x) + \epsilon$ and lower boundary $f(x) - \epsilon$.
2. Plot the regression line, both ϵ boundaries and the actual observations.
3. Print `svr.support_` and mark these observations on your plot.
4. Which observations fall strictly inside the tube, and which sit on or outside its boundary?

```
upper = svr.predict(X) + svr.epsilon
lower = svr.predict(X) - svr.epsilon
inside = np.abs(y - svr.predict(X)) < svr.epsilon
```

Strictly inside: D only.

On or outside: A, B, C, E, F.

`svr.support_` gives support vectors B, C, E, F for this fit.

C.3 – ϵ -Insensitive Loss

$$L_{\epsilon}(y, f(x)) = \max(0, |y - f(x)| - \epsilon)$$

Task 13 – ϵ -Insensitive Loss from the Fitted Model

1. Use `svr.predict(X)` as $f(x)$ for every observation.
2. Calculate the ϵ -insensitive loss $L_{\epsilon} = \max(0, |y - f(x)| - \epsilon)$ for each observation, one line of NumPy, no loop needed.

3. Display Actual, Predicted, Absolute Error and Loss for each observation.
4. Which observations produce zero loss? How does that compare with `svr.support_` from Task 12?

```
preds = svr.predict(X)
loss = np.maximum(0, np.abs(y - preds) - svr.epsilon)
```

Zero loss: A, C, D, E, F.

Only B has positive loss.

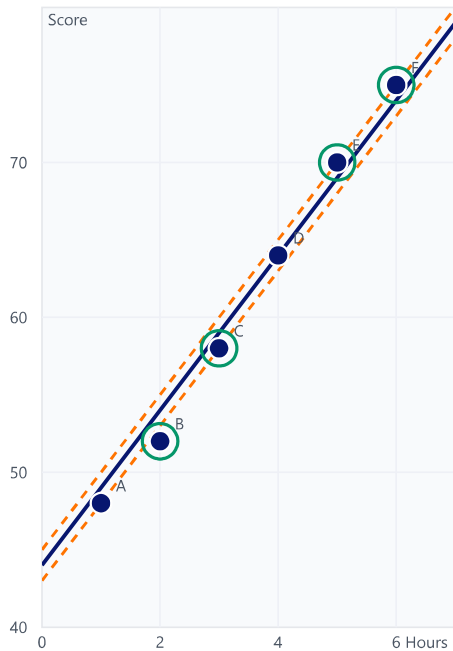
Support vectors are B, C, E, F. Some support vectors can have zero loss if they lie exactly on the tube boundary.

C.4 – Interactive Sandbox: The Effect of ϵ on the Tube

What ϵ Controls

ϵ is the width of the no-penalty tube around the fitted line in SVR. A **small ϵ** forces the line to pass close to almost every point, so nearly all of them end up as support vectors; a **large ϵ** lets many points sit comfortably inside a wide tube with zero loss, leaving fewer points to constrain the line, so it is free to become flatter. Drag the slider below to watch the fitted line, both tube boundaries and the set of support vectors change, computed from a real `SVR(kernel='linear', C=1.0, epsilon=...)` fit on the exact dataset from C.1.

Task 14 – The Effect of ϵ on the Tube



ϵ	1
Slope (w)	5.00
Support vectors	B, C, E, F

Observation (ringed points are support vectors, on or outside the tube)

Shown at $\epsilon = 1$ (a representative middle value). Use the slider on screen to see other values of ϵ .

1. Slide ϵ from its smallest to its largest value. What happens to the width of the tube, and why?
2. How does the number of support vectors change as ϵ increases? Connect this to which points fall inside versus outside the tube.
3. The fitted line keeps getting flatter between $\epsilon = 1$ and $\epsilon = 3$, even though the number of support vectors stays the same. Why does the line keep changing?
4. If this dataset contained a genuine outlier, would a larger or a smaller ϵ make the fitted line less sensitive to it? Explain.

As ϵ increases, the tube becomes wider. This allows more points to fall inside the tube with zero loss.

As ϵ increases, the number of support vectors generally decreases, because fewer points lie outside or on the tube boundary.

The line can still change even if the number of support vectors stays the same, because the positions of the tube boundaries and the dual weights of the support vectors change with ϵ .

If the dataset contains a genuine outlier, a larger ϵ makes the fitted line less sensitive to it, because the outlier can fall inside the wider tube.

Part D

Model Comparison & Understanding

Task 15 – Compare the Models

Property	Linear Regression	SVM	SVR
Target type			
Main purpose			
Output			
Main loss/error concept			
Margin / ϵ -tube			
Role of support vectors			

1. Why can Linear Regression not be directly used for the classification problem in Part B?
2. What is the main conceptual difference between SVM and SVR?
3. Why is feature scaling important when using SVM and SVR?
4. Give one suitable real-world application for Linear Regression, SVM and SVR.

1.

Linear Regression cannot be directly used for classification in Part B because it predicts continuous values, not class labels. Thresholding its output is arbitrary and does not optimise class separation or margin.

2.

SVM is for classification: it separates classes with a maximum margin. SVR is for regression: it fits a continuous function with an ϵ -insensitive tube.

3.

Feature scaling is important for SVM/SVR because they rely on distances and dot products. Unscaled features can dominate the margin or kernel calculations.

4.

Linear Regression: predicting house prices from size.

SVM: email spam classification.

SVR: predicting stock prices or temperature.

End of Lab Sheet 06

Ensure that your notebook contains the completed from-scratch implementations, requested plots, observations, metric values, predictions and the Scikit-learn verification results.



FACULTY OF COMPUTING